

Neural Rendering

Topical research themes 2 – Differentiable Programming

Week 7

assist. prof. dr. Ciril Bohak

Overview

- Neural vs. Differentiable Rendering
- Selected topics:
 - Reduce rendering noise:
 - In MCMC rendering
 - In image-based rendering
 - Realistic images from approximations
 - Generate novel views
 - From multiview input
 - Reshade and relight

Neural Rendering

Neural vs. Differentiable Rendering

- Differentiable rendering
 - makes (relatively conventional) renderer differentiable
 - usually used to support inference (shape from single image, etc.)
- Neural rendering
 - use neural networks at various points in the rendering process
 - lots of methods
 - no real consensus on what a neural rendering process looks like

Realistic Image Synthesis

■ Photorealistic Rendering (CG)

○ Cons:

- Requires lots of manual work
 - Building of high-quality assets
 - Setting up the scenes
- Long rendering times

○ Pros:

- Full control of scene parameters

■ Generative Machine Learning (ML)

○ Cons:

- Requires lots of training data
- No fine-grained semantic control of the scene parameters (e.g., motion or illumination)

○ Pros:

- Fully automatic training
- Interactive inference/rendering

■ Fusion of classical CG components with generative ML

Motivation

- Creating photorealistic assets is challenging using classical techniques
 - Transparency, glossy materials, thin structures (fur, hair)
 - Subsurface scattering: skin, cloth, faces



Lombardi et al. 2019

Neural Rendering – definition

- Definition:

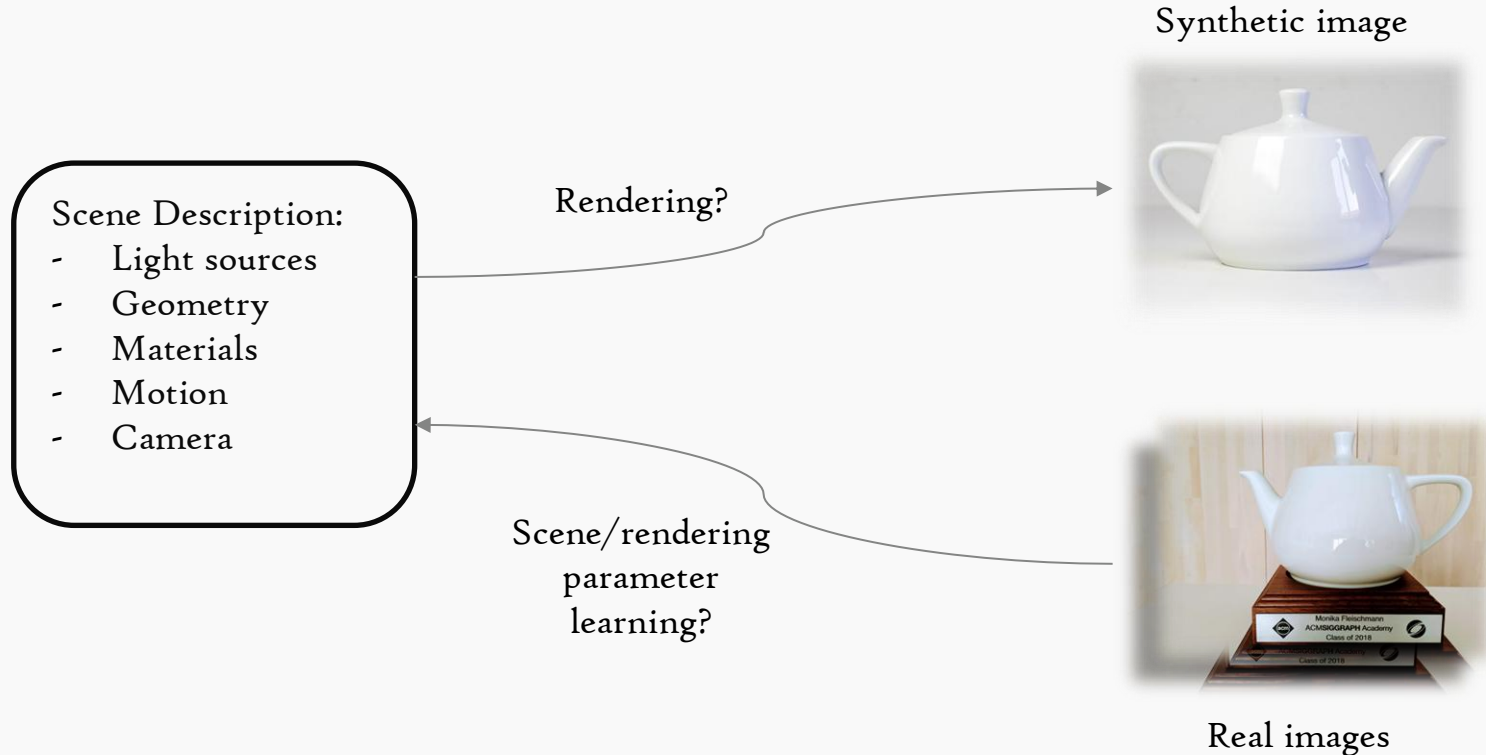
*Deep neural networks for **image or video generation** that enable **explicit or implicit control** of **scene parameters**.*

Generative networks that synthesizes raw pixel output.

Controllable by interpretable parameters or by video/audio input.

Illumination, camera, pose, geometry, appearance, or semantic structure controllable.

Neural Rendering – Graphics vs. Learning



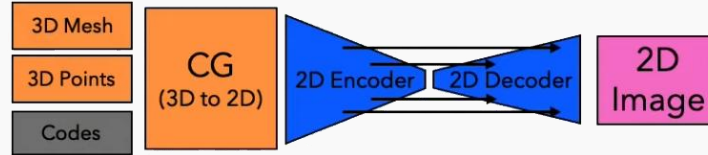
Neural rendering – Approaches

Regression/
Generation



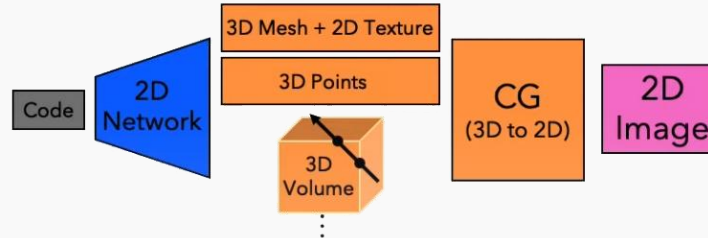
e.g., GQN

Make it more
realistic



e.g., DVP or DNP

Generate and
render



e.g., Neural Volumes

Step, sample
& blend



e.g., NeRF

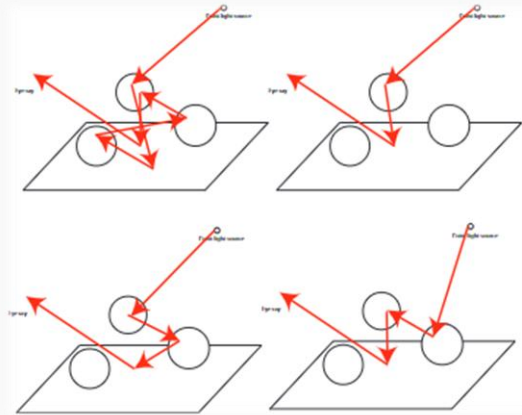
Reducing noise

Reducing noise: MCMC rendering

- Issue:
 - Physically accurate rendering requires tracing very large numbers of complex paths
 - The resulting estimates can have quite high noise
 - Reducing noise by tracing more paths is impractical ($\frac{1}{\sqrt{N}}$.)

- Filtering noisy pixels:

$$\hat{\mathbf{c}}_i = \frac{\sum_{j \in N(i)} d_{i,j} \bar{\mathbf{c}}_j}{\sum_{j \in N(i)} d_{i,j}}$$



Kalantari, Nima Khademi, Steve Bako, and Pradeep Sen. "A machine learning approach for filtering Monte Carlo noise." ACM Trans. Graph. 34, no. 4 (2015): 122-1.

Cross-bilateral filter

$$\hat{\mathbf{c}}_i = \frac{\sum_{j \in N(i)} d_{i,j} \bar{\mathbf{c}}_j}{\sum_{j \in N(i)} d_{i,j}}$$

$$d_{i,j} = e^{-\frac{\text{location} \quad \|\bar{\mathbf{p}}_i - \bar{\mathbf{p}}_j\|^2}{2\alpha_i^2}} \times e^{-\frac{\text{Pixel color} \quad D(\bar{\mathbf{c}}_i, \bar{\mathbf{c}}_j)}{2\beta_i^2}} \times \prod_{k=1}^K e^{-\frac{\text{Features (e.g., which surface, normal, etc.)} \quad D_k(\bar{\mathbf{f}}_{i,k}, \bar{\mathbf{f}}_{j,k})}{2\gamma_{k,i}^2}}$$

Kalantari, Nima Khademi, Steve Bako, and Pradeep Sen. "A machine learning approach for filtering Monte Carlo noise." ACM Trans. Graph. 34, no. 4 (2015): 122-1.

Reducing noise: MCMC rendering

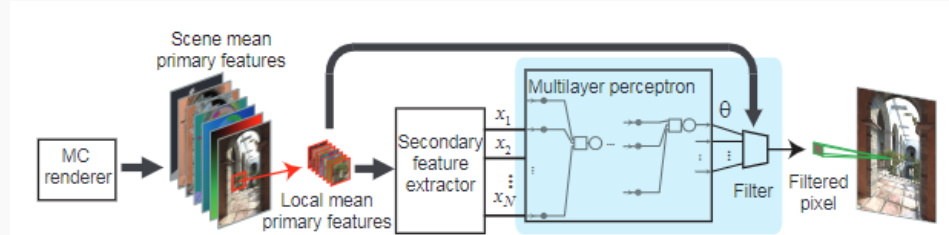


Figure 4: *Our approach combines a standard MLP (Fig. 3) with a matching filter. The local mean primary features (illustrated by a stack of images) contain color, position, and additional features such as world positions, shading normals, etc. A set of secondary features $\{x_1, \dots, x_N\}_i$ (see Sec. 3.3) are extracted from the mean primary features in a local neighborhood of each pixel. The MLP takes the secondary features and outputs the parameters of the filter. The filter then takes the block of mean primary features and outputs a filtered pixel. During training, we minimize the error between the filtered pixel and the ground truth. Once trained, the network can generate appropriate filter parameters for an arbitrary test image.*

Kalantari, Nima Khademi, Steve Bako, and Pradeep Sen. "A machine learning approach for filtering Monte Carlo noise." *ACM Trans. Graph.* 34, no. 4 (2015): 122-1.

Reducing noise: MCMC rendering

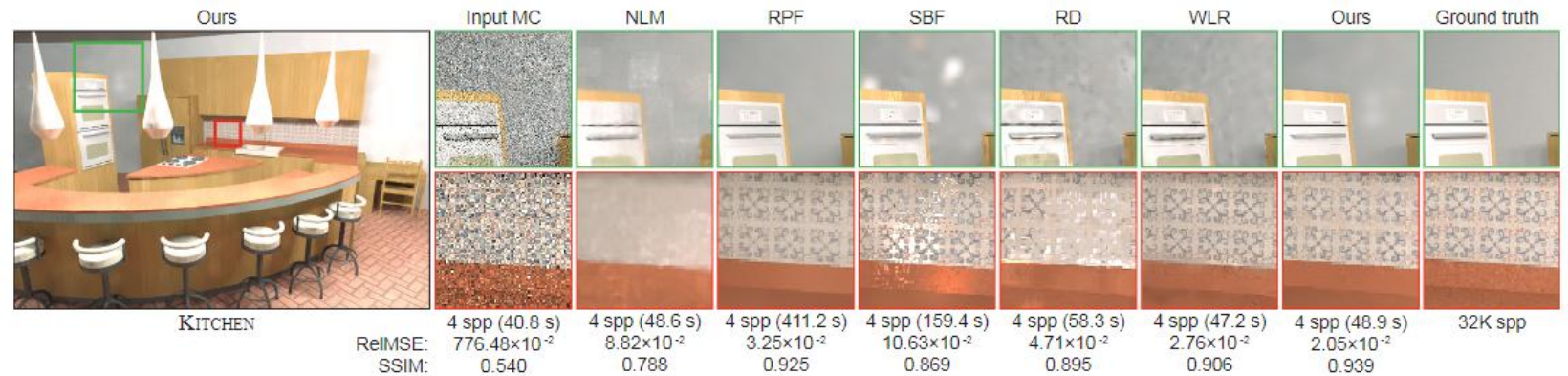


Figure 2: Comparison between our approach and several state-of-the-art algorithms on the KITCHEN scene rendered at 4 spp. Note that the ground truth image is still noisy even at 32K spp. Non-local means filtering (NLM) [Rousselle et al. 2012] is a color-based method which cannot keep geometry or texture detail. Random parameter filtering (RPF) [Sen and Darabi 2012], SURE-based filtering (SBF) [Li et al. 2012], robust denoising (RD) [Rousselle et al. 2013], and weighted local regression (WLR) [Moon et al. 2014] use additional scene features (e.g., world positions) to keep the details. However, they often do not weight the importance of each feature optimally, resulting in under/over blurred regions or splotches in the final result. Our approach preserves scene detail and generates a higher-quality, noise-free result faster than most other methods. The relative mean squared error (RelMSE) and structural similarity (SSIM) index are listed below each image. Larger SSIM values indicate better perceptual quality. Full images are available in the supplementary materials. Scene credit: Jo Ann Elliott.

Kalantari, Nima Khademi, Steve Bako, and Pradeep Sen. "A machine learning approach for filtering Monte Carlo noise." ACM Trans. Graph. 34, no. 4 (2015): 122-1.

See also

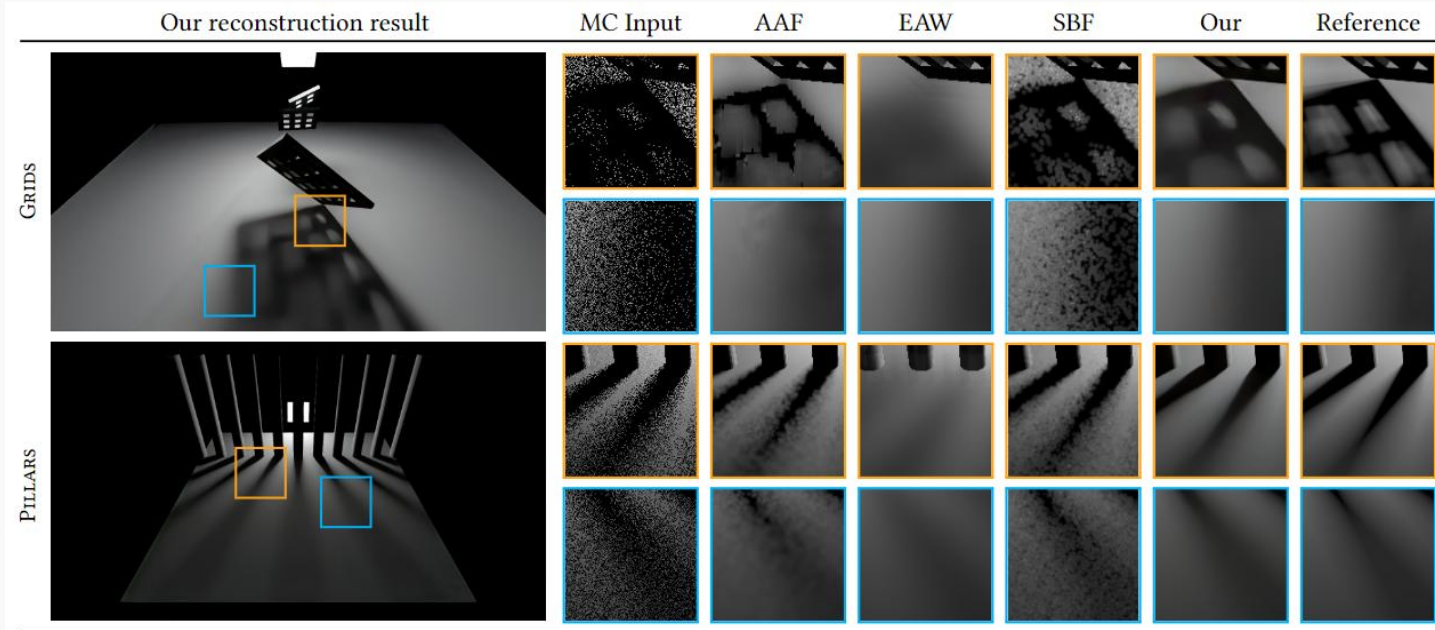


Fig. 10. Closeups for shadow filtering for 1 spp input (MC), axis-aligned filter (AAF), Å-Trous wavelet filter (EAW), SURE-based filter (SBF), and our result.

Chaitanya, Chakravarty R. Alla, Anton S. Kaplanyan, Christoph Schied, Marco Salvi, Aaron Lefohn, Derek Nowrouzezahrai, and Timo Aila. "Interactive reconstruction of Monte Carlo image sequences using a recurrent denoising autoencoder." ACM Transactions on Graphics (TOG) 36, no. 4 (2017): 1-12.

Noise management in IBR

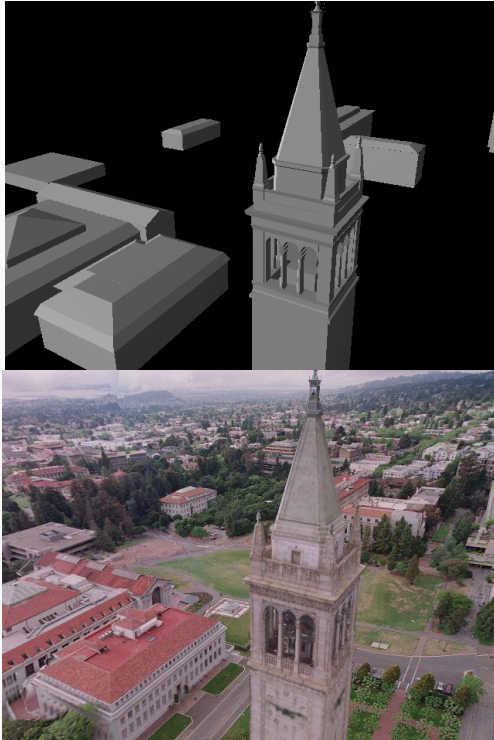
- NeRF could be an example of this
- Image based rendering
 - From several images of a scene, produce a rendering at a new viewport
Typically, using some form of approximate geometric representation
 - Simple cases:
 - SFM yields cameras, blend on a common plane (Photo tourism, Snavely et al. 2006)
 - <https://www.youtube.com/watch?v=mTBPGuPLI5Y>
 - Blending can look poor, texture slides
 - SFM yields points → parametric model, texture from image, render (Façade, Debevec 1996, 1997)
 - https://www.youtube.com/watch?v=RPhGEiM_6lM
 - Many things remain hard to model
 - Errors in recovered model lead to texture problems
 - https://www.youtube.com/watch?v=BHEXm_yQeeg

[1] Snavely, Noah, Steven M. Seitz, and Richard Szeliski. "Photo tourism: exploring photo collections in 3D." In ACM siggraph 2006 papers, pp. 835-846. 2006.

[2] Taylor, Camillo J., Paul E. Debevec, and Jitendra Malik. "Reconstructing polyhedral models of architectural scenes from photographs." In Computer Vision – ECCV'96: 4th European Conference on Computer Vision Cambridge, UK, April 15–18, 1996 Proceedings Volume II 4, pp. 659-668. Springer Berlin Heidelberg, 1996.

[3] Debevec, Paul E. "FACADE: modeling and rendering architecture from photographs and the campanile model." In ACM SIGGRAPH 97 Visual Proceedings: The art and interdisciplinary programs of SIGGRAPH'97, p. 254. 1997.

Noise management in IBR



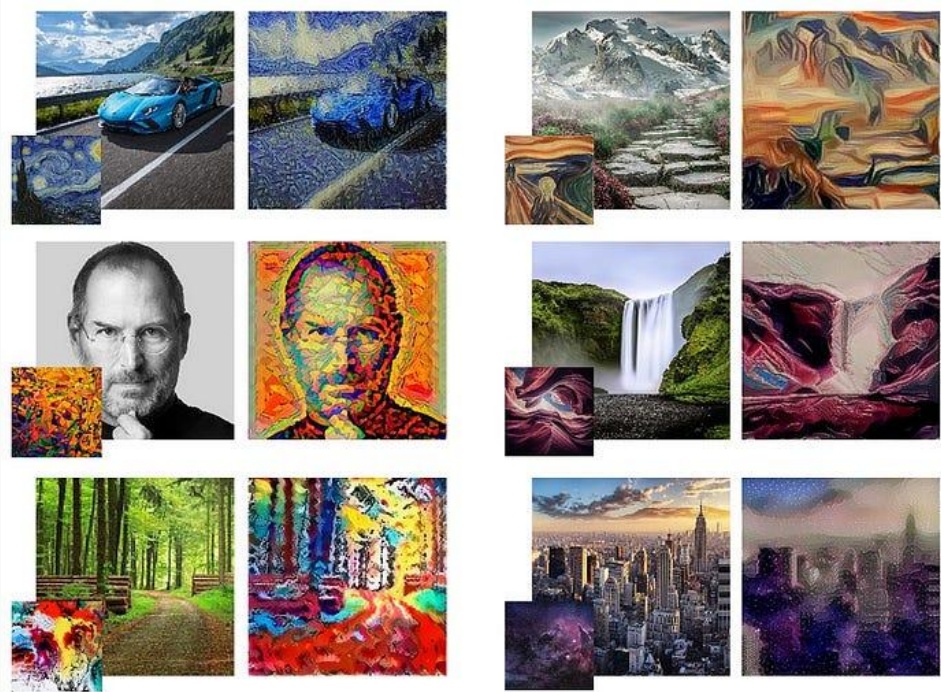
<https://www.pauldebevec.com/Campanile/#movie>

Realistic images from approximations

Why?

- Neural Style transfer
- Colorization
- Super-resolution
- Images from labels

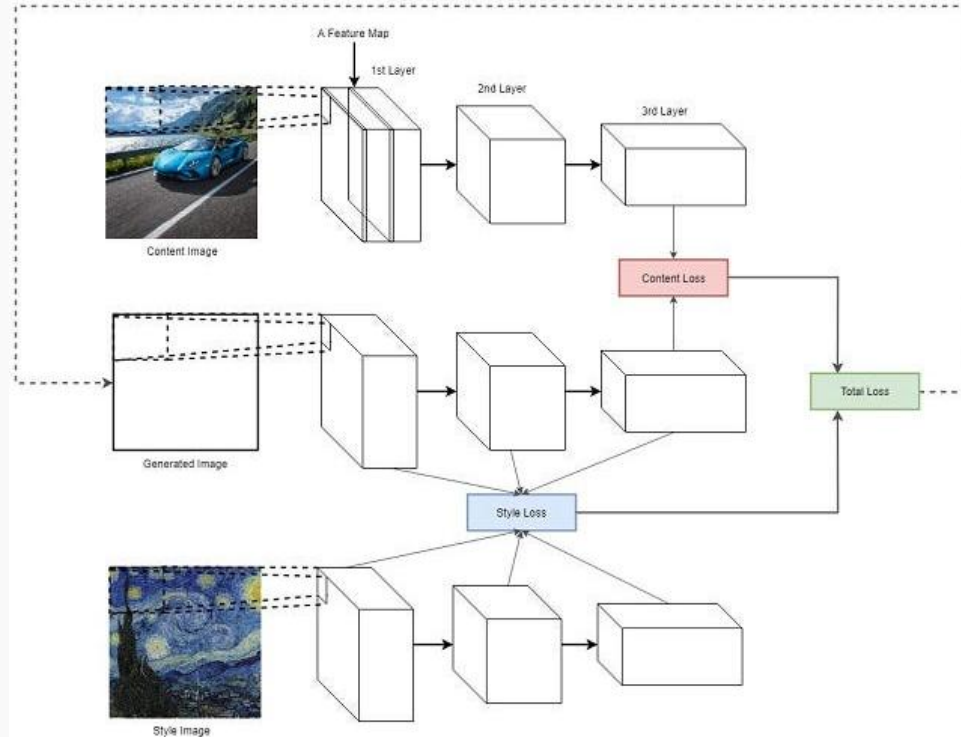
Neural Style transfer



source: Towards Data Science

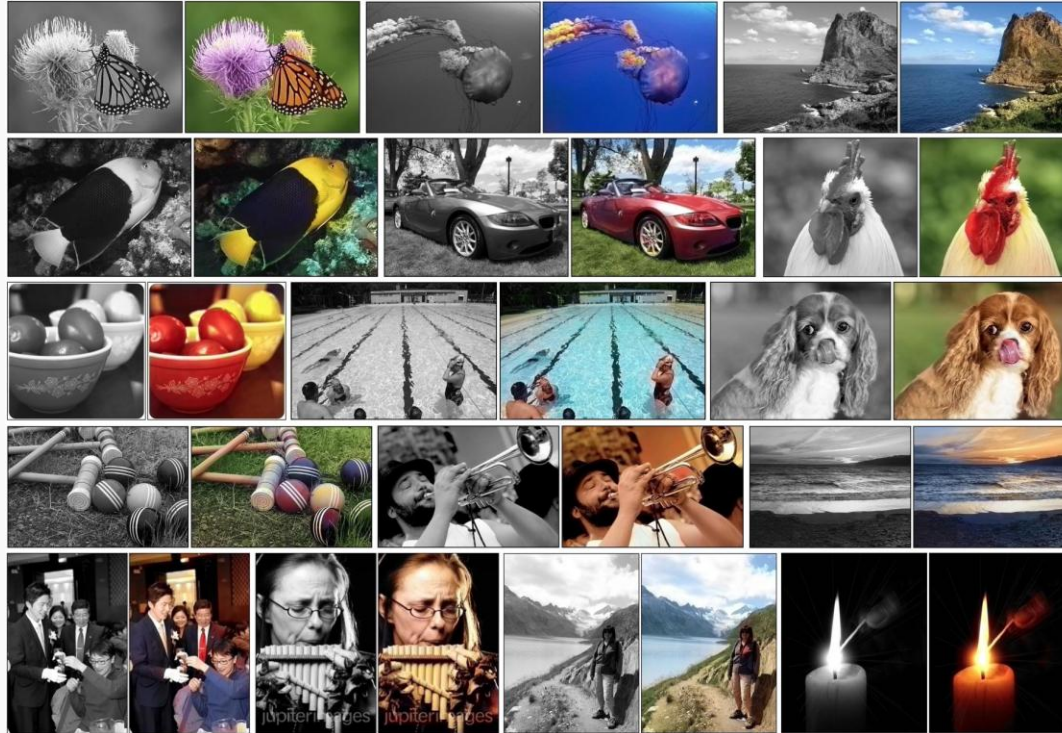
Gatys, Leon, Alexander Ecker, and Matthias Bethge. "A Neural Algorithm of Artistic Style." Journal of Vision 16, no. 12 (2016): 326-326.

Neural Style Transfer



source: Towards Data Science

Colorization



source: Zhang et al. 2016

Žeger, Ivana, Sonja Grgic, Josip Vuković, and Gordan Šišul. "Grayscale image colorization methods: Overview and evaluation." IEEE access 9 (2021): 113326-113346.

Super-resolution

- It is often joined with denoising
- Real-time and non-real-time usage:
 - Games (DLSS, FSR)
 - Photo enhancement (Adobe, Topaz Labs, ...)
- Mostly based on U-Net-based architecture for images
 - Additional recurrent properties for time coherency (video, games)
 - Recently extended to transformer models (DLSS 4)

Images from labels

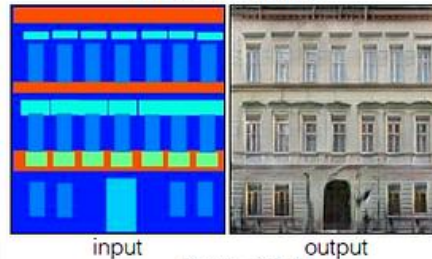
Labels to Street Scene



Aerial to Map



Labels to Facade



Day to Night



BW to Color



Edges to Photo



NeRFs

Rendering Background

- Ray casting

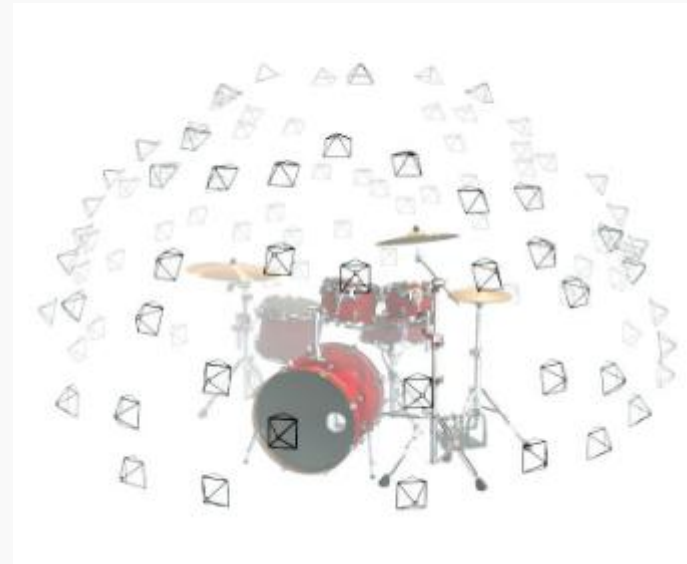
Overview

- A method for synthesizing novel views of complex scenes by optimizing an underlying continuous volumetric scene function using a sparse set of input views.
- Input:
 - Multiple images
 - Known camera positions
- Method:
 - Optimization of continuous volumetric scene function (NN)
- Rendering:
 - Ray casting through space, querying the NN for color and density using 5D input

Input



Input images



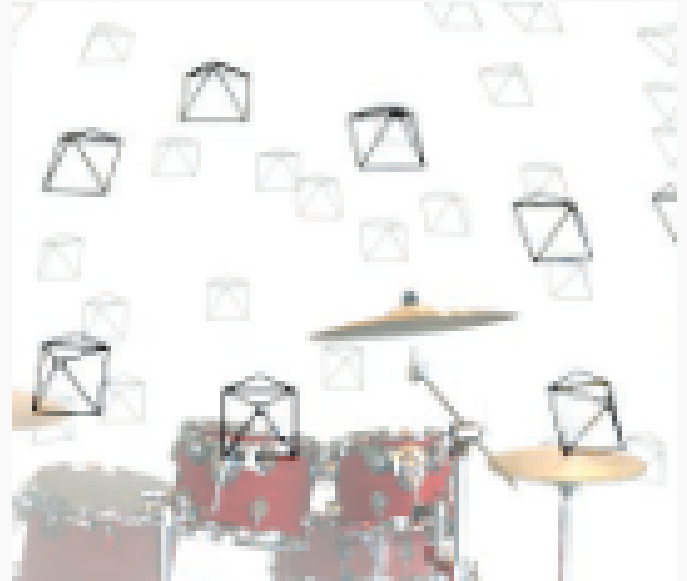
Camera positions

Input

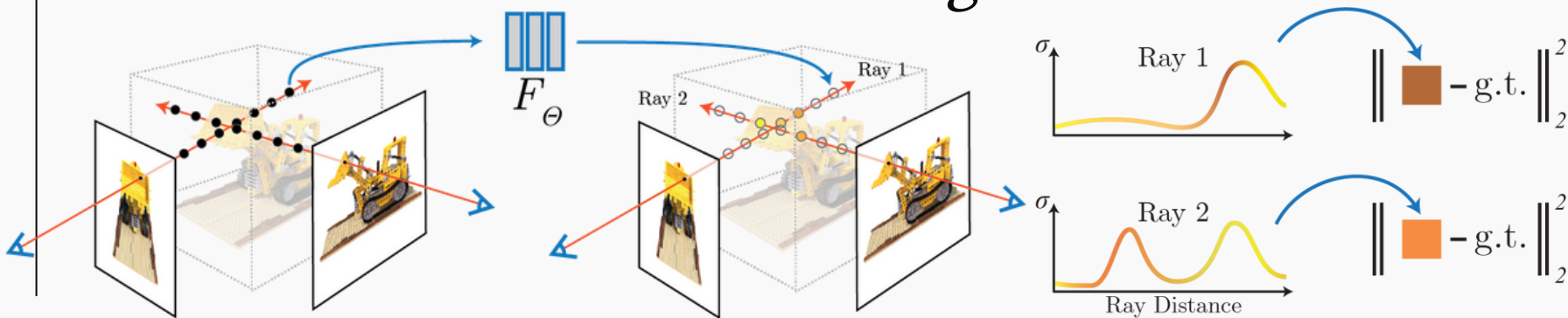
- The camera positions need to be very precise
- We need to know camera (intrinsic) parameters
- We can also register camera images.
 - Poor registration yields bad results:
 - Loss of details
 - Missing information
 - NeRF optimization fails

Scene representation

- 5D vector valued function:
 - 3D location $\mathbf{x} = (x, y, z)$
 - 2D viewing direction $\mathbf{d} = (\theta, \phi)$
- Output:
 - 3D vector, representing emitted color $\mathbf{c} = (r, g, b)$
 - Scalar, representing volume density σ



Rendering



$$C(\mathbf{r}) = \int_{t_n}^{t_f} \mathbf{c}(\mathbf{r}(t), \mathbf{d}) \sigma(\mathbf{r}(t)) e^{-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds} dt$$

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N (1 - e^{-\sigma_i \delta_i}) \mathbf{c}_i e^{-\sum_{j=1}^{i-1} \sigma_j \delta_j}$$

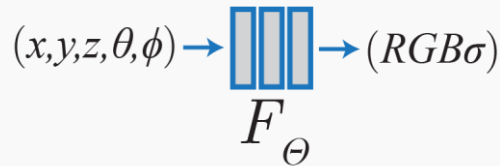
Rendering – differentiability

- Calculating $\hat{C}(\mathbf{r})$ from a set of $(\mathbf{c}_i, \sigma_i)$ values is trivially differentiable.
- It reduces to traditional alpha compositing values:

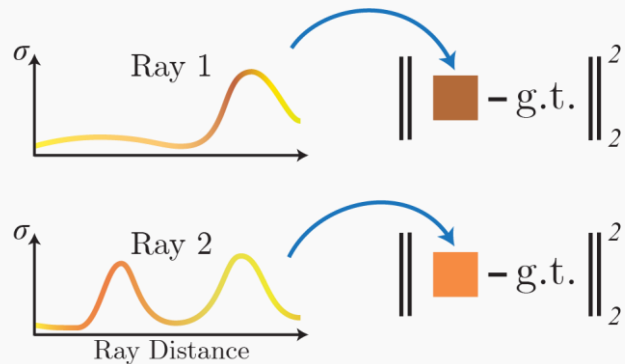
$$\alpha_i = 1 - e^{-\sigma_i \delta_i}$$

Optimization

- Overfit NN to a set of images.
 - More precisely to all the image pixels seen from individual camera positions
- Use MLP as a NN
- Since the whole rendering process is differentiable:
We can use it for MLP optimization.



Volume rendering Rendering Loss



Multiview consistency

- Restricting the network to predict the volume density σ as a function of only location $\mathbf{x}$.
- Allow the RGB color $\mathbf{c}$ to be predicted as function of both location and viewing direction.

Positional encoding

- Neural networks are biased towards learning lower frequency function.
- Mapping the inputs to a higher dimensional space using high frequency functions before passing them to the network enables better fitting of data that contains high frequency variation.
- We map the $xyz\theta\phi$ using:
$$\gamma(p) = (\sin(2^0\pi p), \cos(2^0\pi p), \dots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p))$$
into higher dimensional spaces $L = 10$ for x, y, z , and $L = 4$ for d , where d is cartesian representation of $\theta\phi$.

Hierarchical volume sampling

- Two layers of NN:
 - Coarse:
 - Sample first to get idea where stuff is
 - Fine:
 - Sample additionally in places with high density in the coarse NN.
- They concurrently optimize both networks
- Loss is total squared error between the rendered and true pixel colors for both coarse and fine rendering.

Summary

Pros:

- Great compression!
- Authors claim 5MB instead of 15GB which is a bit overrated, but still it is less then combined size of all input images.

Cons:

- Slow learning
 - 1-2 days on a single V100 (100k – 300k iterations)
- Not the fastest rendering

Many improvements:

- Multiscale representation Mip-NeRF
- Learned initializations
- Relighting
- Speed optimization – InstaNGP
- ...

Gaussian Splatting

NeRF Summary

Pros:

- Great compression!
- Authors claim 5MB instead of 15GB which is a bit overrated, but still it is less than combined size of all input images.

Many improvements:

- Multiscale representation Mip-NeRF
- Learned initializations
- Relighting
- Speed optimization – InstaNGP
- ...

Cons:

- **Slow learning**
 - 1-2 days on a single V100 (100k – 300k iterations)
- **Not the fastest rendering**

Rendering Background

- What is splatting?
- Basic idea comes from *Surface splatting* [1] from 2001, *Point-Based Rendering* [2] from 2004 and *Differentiable Surface Splatting* [3] from 2019.

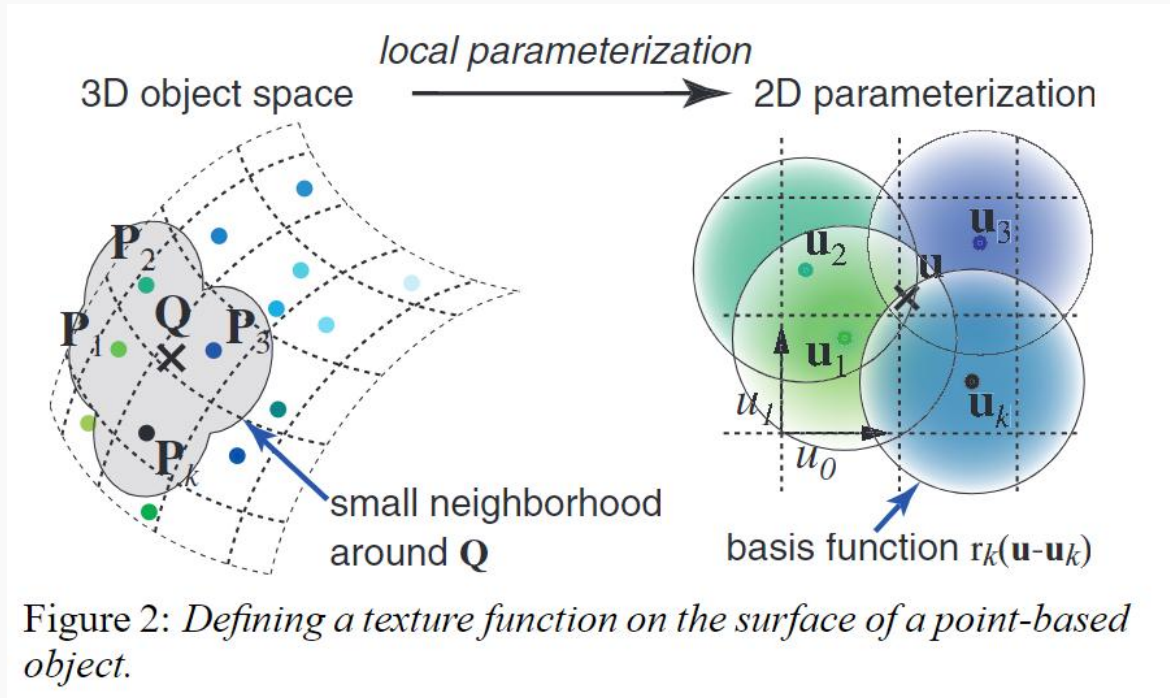
[1] Zwicker, Matthias, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. "Surface splatting." In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pp. 371-378. 2001.

[2] Sainz, Miguel, and Renato Pajarola. "Point-based rendering techniques." *Computers & Graphics* 28, no. 6 (2004): 869-879.

[3] Yifan, Wang, Felice Serena, Shihao Wu, Cengiz Öztireli, and Olga Sorkine-Hornung. "Differentiable surface splatting for point-based geometry processing." *ACM Transactions on Graphics (TOG)* 38, no. 6 (2019): 1-14.

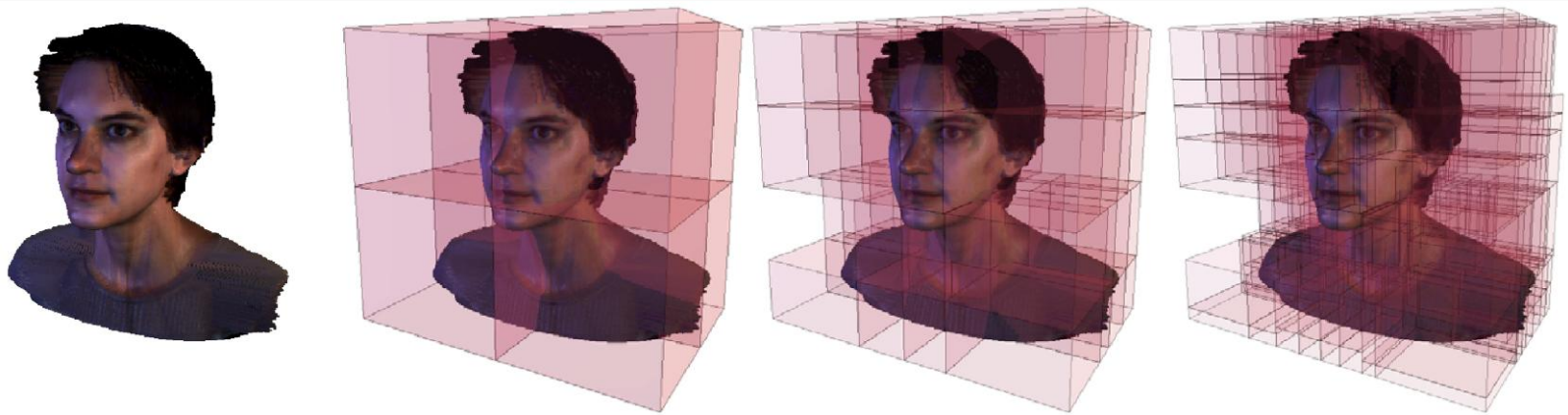
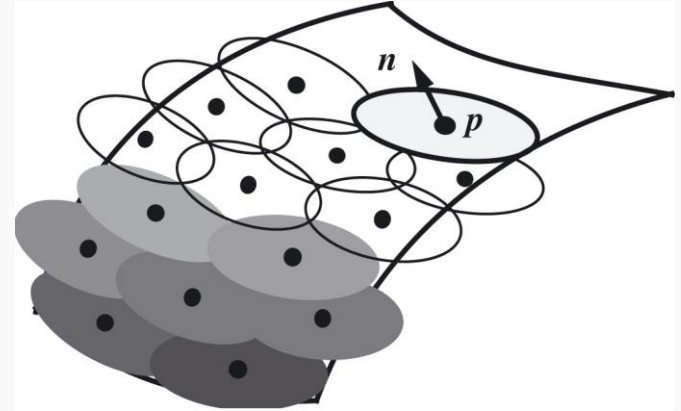
Overview

- Surface Splatting



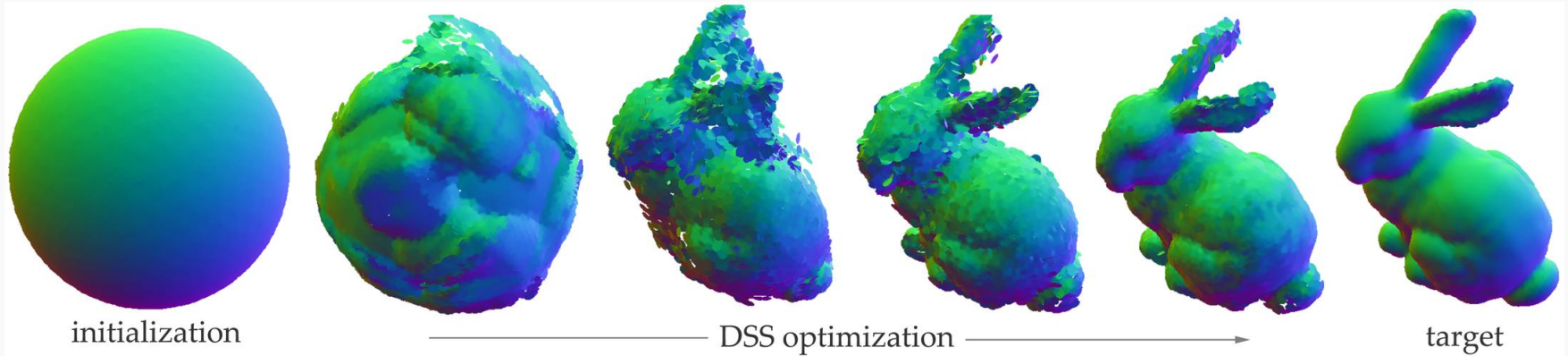
Overview

- Point-Based Rendering



Overview

- Differentiable Surface Splatting



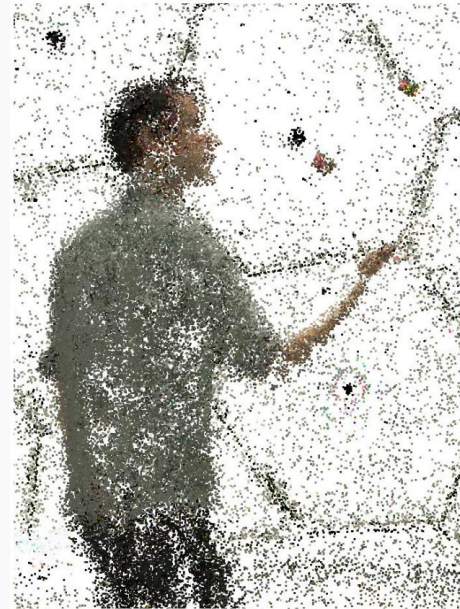
Overview

- A method for synthesizing novel views of complex scenes by optimizing an underlying point cloud representation obtained from images with structure from motion or some similar input.
- Input:
 - Multiple images converted to point cloud representation (typically millions of points)
 - Known camera positions
- Method:
 - Optimizing Gaussian splats in differentiable way
- Rendering:
 - Splatting

Input

Each 3D Gaussian is parametrized by:

- Mean μ interpretable as location x, y, z ;
- Covariance Σ ;
- Opacity $\sigma(\mathbf{a})$, a sigmoid function mapping to interval $[0, 1]$;
- Color parameters, either RGB values or spherical harmonics coefficients.



Source: Luiten et al.

Scene representation

- Covariance is anisotropic, meaning that we can have an ellipsoid rotated and stretched along any direction.
- We need to express it in different form, e.g. eigendecomposition:

$$\Sigma = RSS^T R^T$$

- Two benefits of using Gaussians:
 - Effectively they represent space close to its mean
 - They are defined on the whole 3D space, meaning gradients are defined everywhere, which is great for optimization.

Scene representation

- The impact of 3D Gaussian i on arbitrary 3D point p in 3D space:

$$f_i(p) = \sigma(\alpha_i) e^{-\frac{1}{2}(p-\mu_i)^T \Sigma_i^{-1}(p-\mu_i)}$$

- This is pretty much a probability density function of the multivariate normal distribution with exception of normalization term being ignored and being weighted by the opacity instead.

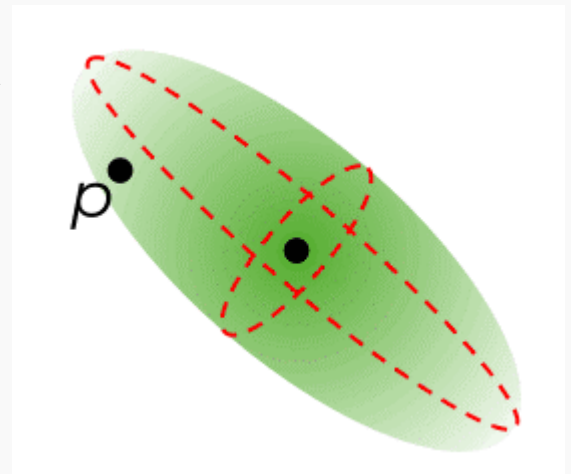


Image Formation

- From NeRFs:

$$\begin{aligned} c(p) &= \sum_{i=1}^N c_i (1 - e^{-\sigma_i \delta_i}) T_i = \sum_{i=1}^N c_i (1 - e^{-\sigma_i \delta_i}) e^{-\sum_{j=1}^{i-1} \sigma_j \delta_j} = \\ &= \sum_{i=1}^N c_i \underbrace{(1 - e^{-\sigma_i \delta_i})}_{\alpha_i} \prod_{j=1}^{i-1} \underbrace{e^{-\sigma_j \delta_j}}_{1 - \alpha_j} = \sum_{i=1}^N c_i \alpha_i \underbrace{\prod_{j=1}^{i-1} (1 - \alpha_j)}_{\text{transmittance}} \end{aligned}$$

- For Gaussians:

$$c(p) = \sum_{i \in N} c_i f_i^{2D}(p) \underbrace{\prod_{j=1}^{i-1} (1 - f_j^{2D}(p))}_{\text{transmittance}}$$

Image Formation

- What is f^{2D} ?
 - It is a projection of $f(p)$ onto an image plane of the camera
 - Both a 3D point representation and its projection are multivariate Gaussians
 - Because of this, Gaussian on a pixel can be computed using the same formula as the impact of 3D Gaussian on other points in 3D. We only need to project mean μ and covariance Σ onto 2D using derivation from EWA splatting [4].
- Transforming means to 2D is trivial:

$$z \begin{bmatrix} u/z \\ v/z \\ 1 \end{bmatrix} = \begin{bmatrix} u \\ v \\ z \end{bmatrix} = KW \begin{bmatrix} \mu_x \\ \mu_y \\ \mu_z \\ 1 \end{bmatrix}$$
$$\mu^{2D} = \begin{bmatrix} \mu_x^{2D} \\ \mu_y^{2D} \end{bmatrix} = \begin{bmatrix} u/z \\ v/z \end{bmatrix}$$

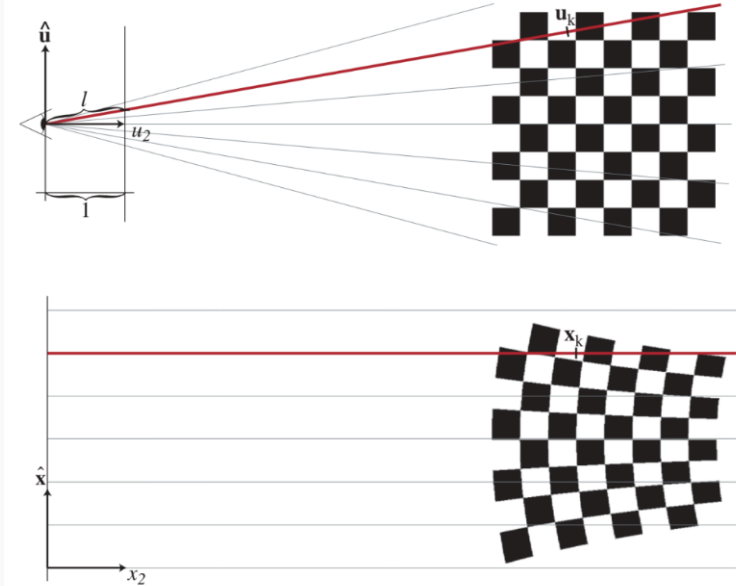
[4] Zwicker, Matthias, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. "EWA volume splatting." In Proceedings Visualization, 2001. VIS'01., pp. 29-538. IEEE, 2001.

Image Formation

- Covariance in 2D is defined using Jacobian of mean:

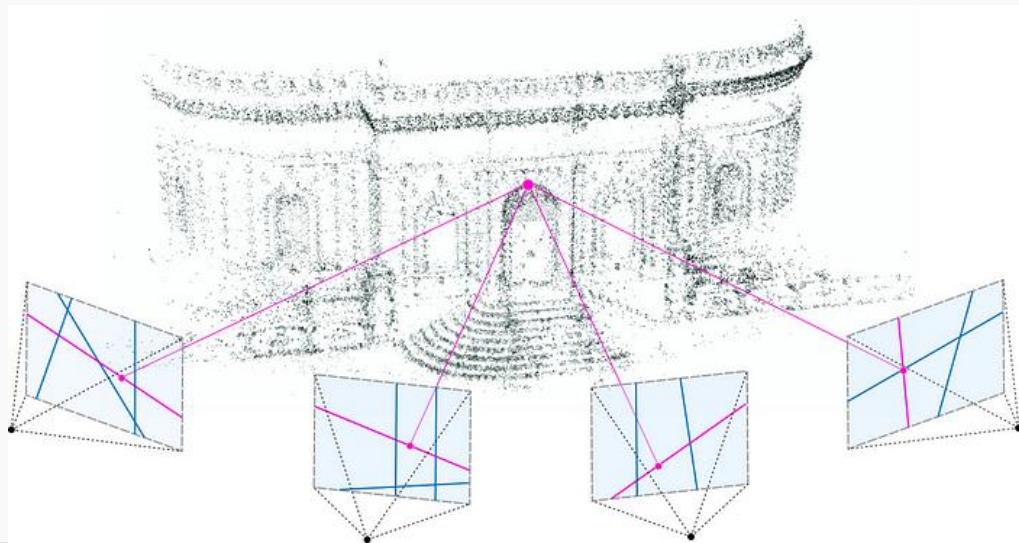
$$J = \frac{\partial \mu^{2D}(\mu)}{\partial \mu}$$
$$\Sigma^{2D} = JW\Sigma W^T J^T$$

- The whole process is differentiable which is crucial for the optimization.



Optimization

- A sparse 3D point cloud produced by SfM with means initialization speeds up the optimization.
- Covariances are initialized as spheres.
- A simple Stochastic Gradient Descent is used for fitting.
- The scene is optimized for a loss function that is a combination of L_1 and D-SSIM between a ground truth view and a current render.

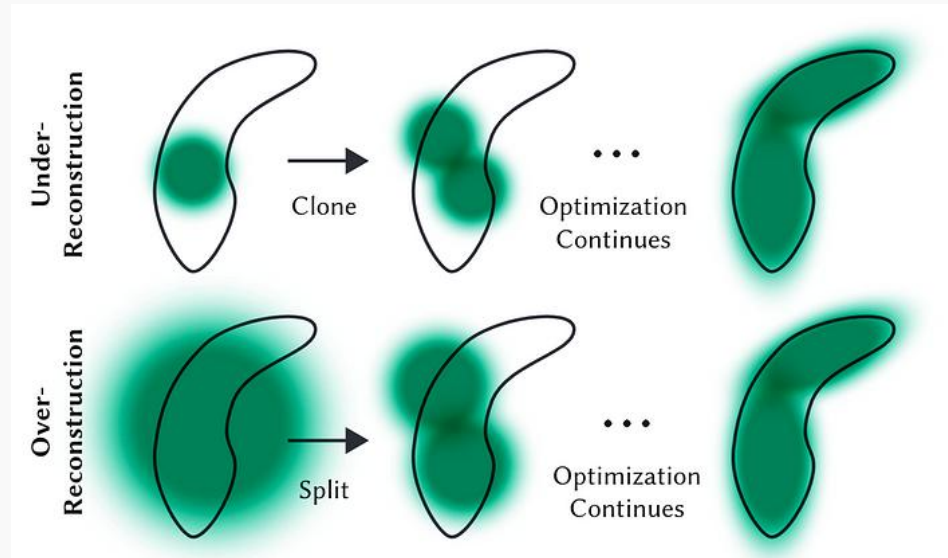


Optimization – algorithm

1. Rasterize the gaussians to an image using differentiable gaussian rasterization (more on that later)
2. Calculate the loss based on the difference between the rasterized image and ground truth image
3. Adjust the gaussian parameters according to the loss
4. Apply automated densification and pruning

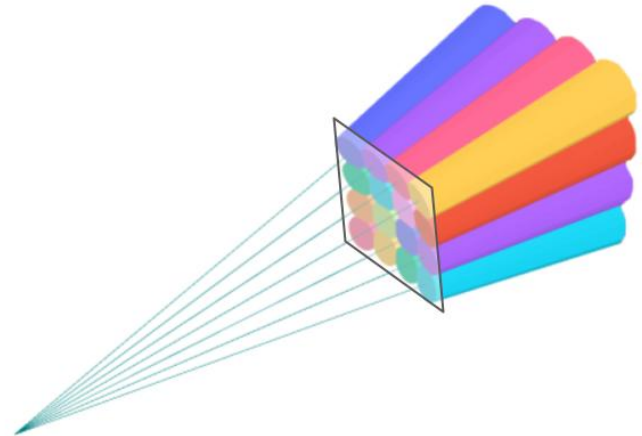
Optimization – densification and pruning

- Adaptive densification is executed every now and then (e.g., 100 steps of SGD) to address under- and over-reconstruction.
- Cloning small gaussians.
- Splitting gaussians with large gradients.
- Removing points converged to very low values of α .



Rendering

- We still need to traverse along the rays through all the pixels.
- However, it is much less computationally demanding:
 - For a camera, $f(p)$ of each 3D point can be projected into 2D in advance, before iteration over pixels.
 - There is no NN inference, 2D Gaussians are blended onto an image directly.
 - There is no need for dense space sampling (as is the case for NeRFs).
 - Gaussian sorting is done once per frame.
- Rendering of each pixel is reduced to α -blending of pre-ordered points from the tile the pixel belongs to.



Rendering

- View-dependent colors with Spherical Harmonics.

$$\begin{array}{ccccccccc}
 & m = -2 & m = -1 & m = 0 & m = 1 & m = 2 & & & \\
 \ell = 0 & & & c_1 \cdot \text{blue sphere} & + & & & & \\
 \ell = 1 & & + c_2 \cdot \text{red sphere} & + c_3 \cdot \text{rainbow sphere} & + c_4 \cdot \text{rainbow sphere} & + & & & \\
 \ell = 2 & + c_5 \cdot \text{rainbow sphere} & + c_6 \cdot \text{rainbow sphere} & + c_7 \cdot \text{rainbow sphere} & + c_8 \cdot \text{rainbow sphere} & + c_9 \cdot \text{rainbow sphere} & = & \text{rainbow sphere} & \\
 \text{red} = \text{sigm} & (& \text{rainbow sphere} & \theta, \varphi &) \cdot 255
 \end{array}$$

Summary

Pros:

- Very fast!
- Works great for thin features.
- Removes floaters.
- Fast model optimization (1 – few hours).

Cons:

- Not physically based.
- Cannot be simply relighted.

Many improvements:

- <https://github.com/MrNeRF/awesome-3D-gaussian-splatting>
- ...